ChurnGuard:
CUSTOMER CHURN PREDICTION ONLINE APP

End-to-End MLOps Application

Final Project Report

Building an AI Application with MLOps

Field	Details
Student Name	Nikesh Kumar Mandal
Roll Number	ID25M805
Institution	Indian Institute of Technology Madras
Course	MLOPS
Submission Mode	Physical
Date:	April 28, 2026

1. Executive Summary

ChurnGuard is a production-grade, MLOps-compliant AI application that predicts whether a telecom customer is likely to cancel their subscription (churn). The system is built as a complete end-to-end solution covering data ingestion, feature engineering, model training, REST API serving, a web frontend, experiment tracking, pipeline orchestration, and real-time monitoring.

The application is deployed via Docker Compose as six independent containerised services. All MLOps requirements from the course guidelines are implemented: DVC for pipeline and data versioning, MLflow for experiment tracking, Airflow for pipeline orchestration, Prometheus and Grafana for monitoring, FastAPI for model serving, and Streamlit for the user interface.

2. Problem Statement & Objectives

2.1 Business Problem

Customer churn — when a subscriber cancels their service — is one of the costliest problems in subscription-based industries. Research shows that acquiring a new customer costs 5–7× more than retaining an existing one. A 5% improvement in retention can increase profits by 25–95%.

Without a predictive system, retention teams rely on gut feel and simple rules. These miss complex multi-factor patterns. A machine learning model learns from historical behaviour to identify at-risk customers before they leave, enabling targeted, cost-effective interventions.

2.2 ML Objective

Build a binary classification model that predicts churn (Yes/No) from customer profile data. The model is served via a REST API that accepts customer features and returns a probability score, risk level, and recommended action.

2.3 Success Metrics

Metric Type	Metric	Target	Achieved
ML	F1-Score (test set)	≥ 0.52	0.525
ML	ROC-AUC (test set)	≥ 0.60	0.665
Business	Inference Latency	< 200 ms	< 5 ms
Business	API Error Rate	< 5%	< 1%
MLOps	All pipeline stages automated	Yes	Yes
MLOps	All services containerised	Yes	Yes

3. System Architecture

3.1 High-Level Design

The system follows a strict microservices architecture with loose coupling between all components. The frontend communicates with the backend exclusively via REST API calls — they share no code, no database, and no filesystem access. This is the fundamental design principle required by the MLOps guidelines.

Layer	Component	Technology	Host Port
User Interface	Streamlit Web App	Python / Streamlit	8501
Inference	REST API	Python / FastAPI	8005
Experiment Tracking	MLflow Server	MLflow + SQLite	5005
Monitoring	Metrics Scraper	Prometheus	9090
Monitoring	Dashboard	Grafana	3005
Orchestration	Pipeline Scheduler	Apache Airflow	8081

3.2 Service Communication

- Frontend (Streamlit) → API (FastAPI): HTTP POST /predict, GET /health, GET /model/info
- Prometheus → API: HTTP GET /metrics every 15 seconds
- Grafana → Prometheus: PromQL queries every 10 seconds
- Airflow → Pipeline scripts: Python subprocess via PythonOperator
- All services communicate within a private Docker bridge network (churn_net)

3.3 Data Flow

Raw CSV (telco_churn.csv) → DataIngestionPipeline [validate + baseline stats] → FeatureEngineeringPipeline [encode + scale + split] → TrainingPipeline [GridSearchCV + threshold tuning + MLflow] → Artifacts (model.pkl, scaler.pkl, encoders.pkl) → FastAPI loads at startup → CustomerInput JSON → preprocess → predict → PredictionResponse JSON → Streamlit renders result.

4. Implementation Details

4.1 Dataset

A synthetic Telco customer dataset of 5,000 rows and 21 features was generated to simulate real-world churn patterns. The dataset has a churn rate of ~32%, creating a class imbalance that is addressed through class_weight='balanced' in model training and SMOTE oversampling in version 2.

Feature Category	Features
Account	tenure, monthly_charges, total_charges, num_products, contract_type, payment_method, paperless_billing
Services	internet_service, online_security, tech_support, streaming_tv, streaming_movies, device_protection, online_backup, phone_service, multiple_lines
Demographics	gender, senior_citizen, partner, dependents
Engineered	charges_per_tenure, value_score

4.2 Data Ingestion Pipeline

The DataIngestionPipeline class in src/pipeline/data_ingestion.py performs: (1) CSV loading with FileNotFoundError on missing data, (2) schema validation raising ValueError on missing columns, (3) automated quality checks for missing values, duplicates, and impossible values, (4) statistical baseline computation (mean, std, percentiles for all numeric features; value distributions for categorical features) saved to config/baseline_stats.json for later drift detection, (5) validated data saved to data/processed/.

4.3 Feature Engineering Pipeline

The FeatureEngineeringPipeline class performs: label encoding of all 16 categorical features with LabelEncoder (fitted on training data, applied to validation and test), StandardScaler normalisation of all numeric features, creation of two derived features (charges_per_tenure = monthly_charges / (tenure+1), value_score = total_charges / (monthly_charges+1)), and stratified 70/15/15 train/val/test split. All fitted transformers are saved as pickle artifacts to ensure identical preprocessing at inference time.

4.4 Model Training (Version 2)

Version 2 introduced GridSearchCV with 5-fold stratified cross-validation across four model families, SMOTE oversampling on the training set, and probability threshold optimisation.

Model	Best Val F1	Best Val ROC-AUC	Training Time
Logistic Regression	0.523	0.671	< 1s
Random Forest (v1)	0.547	0.686	1.4s
Gradient Boosting (v1)	0.537	0.685	2.0s
Logistic Regression (v2 tuned)	0.530	0.673	8s
Random Forest (v2 tuned)	0.556+	0.690+	~5 min
Gradient Boosting (v2 tuned)	0.540+	0.688+	~8 min
SVM (v2)	0.510	0.660	~3 min

4.5 Model Serving (FastAPI)

The FastAPI backend exposes 8 endpoints. At startup it loads all pickle artifacts. Each prediction request: validates input with Pydantic, applies identical preprocessing, uses the optimised threshold from threshold.json (not hardcoded 0.5), returns churn probability + risk level + inference time. The /metrics endpoint serves Prometheus-compatible metrics.

4.6 Frontend (Streamlit)

The Streamlit application has five pages: Single Prediction (form-based), Batch Prediction (CSV upload + download), Model Info (metadata + feature importance chart), Monitoring (live health, readiness, drift report), and User Manual. The frontend has no Python ML dependency — it communicates exclusively via HTTP calls to the FastAPI backend.

5. MLOps Implementation

5.1 Version Control — Git + DVC

Git tracks all source code. DVC tracks large binary files (dataset, model artifacts, preprocessor pickles) that cannot be stored efficiently in Git. The dvc.yaml defines the pipeline as a 3-stage DAG: ingest → featurize → train. Running dvc repro re-executes only stages whose inputs have changed.

- git add / commit: all source code, configs, dvc.yaml, params.yaml
- dvc add: data/raw/telco_churn.csv, src/api/artifacts/best_model.pkl, scaler.pkl, encoders.pkl
- dvc repro: reproduces the full pipeline from raw data to trained model
- dvc metrics show: displays test F1, ROC-AUC, model name from metrics.json

5.2 Experiment Tracking — MLflow

Every training run is logged to MLflow with: run name, version tag, hyperparameters, all six metrics (accuracy, F1, precision, recall, ROC-AUC, training time), the fitted model as an artifact, and the model registered in the MLflow Model Registry. The Docker MLflow instance reads from the mounted ./mlruns folder at localhost:5005.

MLflow Entity	What is tracked
Parameters	n_estimators, max_depth, learning_rate, C, threshold, smote flag
Metrics	accuracy, f1, precision, recall, roc_auc, cv_f1, training_time_s
Artifacts	Serialised model pickle, feature importance JSON
Tags	version (v1/v2), model_family, threshold_tuned, smote
Registry	4 registered models (LR, RF, GB, SVM) each with version history

5.3 Pipeline Orchestration — Apache Airflow

The churn_prediction_pipeline DAG runs on a @daily schedule with four tasks: data_ingestion → feature_engineering → model_training → api_health_check. Each PythonOperator task dynamically adds /opt/airflow/dags to sys.path so pipeline modules can be imported by bare name. The final BashOperator task calls curl http://api:8000/ready via the Docker internal network to confirm the model loaded after training.

Key fix applied: the Airflow Docker image was extended with a custom Dockerfile.airflow that pre-installs scikit-learn==1.3.2, pandas==2.0.3, numpy==1.24.4, imbalanced-learn==0.11.0, scipy==1.10.1, and matplotlib==3.7.5 — all pinned to the last versions supporting Python 3.8 (the version bundled with apache/airflow:2.8.1).

5.4 Monitoring — Prometheus + Grafana

The FastAPI /metrics endpoint exposes five Prometheus metrics scraped every 15 seconds:

Metric	Type	Purpose
churn_predictions_total	Counter	Total predictions split by result (churn/no_churn)

Metric	Type	Purpose
churn_prediction_latency_seconds	Histogram	End-to-end inference latency distribution
churn_api_errors_total	Counter	API errors by endpoint for SLA monitoring
churn_active_requests	Gauge	Live concurrency tracking
churn_drift_detected	Gauge	Binary drift flag (1=drift, 0=normal)

Grafana (localhost:3005) auto-loads a pre-provisioned dashboard showing prediction rate, p99 latency, error count, active requests, and drift status — all in near-real-time with a 10-second refresh.

5.5 Data Drift Detection

At training time, baseline_stats.json records the mean, std, and percentiles of every numeric feature. The /monitoring/drift endpoint collects the last 500 prediction inputs and runs a Kolmogorov-Smirnov test per feature comparing the live distribution against a synthetic sample from the training baseline. A p-value < 0.05 flags that feature as drifted and sets the Prometheus churn_drift_detected gauge to 1, triggering a Grafana alert.

5.6 Containerisation — Docker Compose

Six services are defined in docker-compose.yml within a shared churn_net bridge network. The API healthcheck uses the internal container port (8000), not the host-mapped port (8005). The frontend declares depends_on: api: condition: service_healthy so it only starts after the model is confirmed loaded.

6. Test Plan & Test Report

6.1 Acceptance Criteria

Criterion	Target	Result
All unit test cases pass	100% (8/8)	✓ PASS
Inference latency	< 200 ms	✓ < 5 ms
Test-set F1 Score	≥ 0.52	✓ 0.525
Test-set ROC-AUC	≥ 0.60	✓ 0.665
/health endpoint returns 200	200 OK	✓ PASS
/ready endpoint returns 200	200 OK when model loaded	✓ PASS

6.2 Test Cases

ID	Module	Description	Result
TC-01	DataIngestionPipeline	Valid CSV loads correctly and passes schema validation	PASS
TC-02	DataIngestionPipeline	Missing required columns raise ValueError with informative message	PASS
TC-03	FeatureEngineeringPipeline	engineer_features adds charges_per_tenure and value_score columns	PASS
TC-04	FeatureEngineeringPipeline	charges_per_tenure = monthly_charges / (tenure+1) exactly	PASS
TC-05	Model	predict() returns binary value 0 or 1	PASS
TC-06	Model	predict_proba() returns value in range [0.0, 1.0]	PASS
TC-07	DriftDetector	detect() always returns dict with drift_detected key	PASS
TC-08	API Latency	Preprocessing + inference completes in < 200 ms (SLA)	PASS

7. Low-Level Design — API Endpoint Specifications

7.1 Base URL

http://localhost:8005 (host-mapped port; internal container port is 8000)

7.2 Endpoints

GET /health

GET /ready

POST /predict

POST /predict/batch

GET /model/info

GET /model/features

GET /monitoring/drift

GET /metrics

Endpoint	Method	Purpose	Response Code
/health	GET	Liveness probe — confirms API process is alive	200
/ready	GET	Readiness probe — confirms model is loaded	200 / 503
/predict	POST	Single customer churn prediction	200 / 422 / 503
/predict/batch	POST	Batch prediction (max 1000 customers)	200 / 400 / 503
/model/info	GET	Model metadata, hyperparams, val metrics	200 / 404
/model/features	GET	Feature importances sorted descending	200 / 404
/monitoring/drift	GET	KS-test drift report for last 500 inputs	200
/metrics	GET	Prometheus scrape endpoint	200

POST /predict — Request Body (CustomerInput)

Field	Type	Required	Constraints
tenure	float	Yes	0 ≤ tenure ≤ 120
monthly_charges	float	Yes	≥ 0
total_charges	float	Yes	≥ 0
num_products	int	Yes	1 ≤ num_products ≤ 10
contract_type	string	Yes	Month-to-month | One year | Two year
payment_method	string	Yes	Electronic check | Mailed check | Bank transfer | Credit card
internet_service	string	Yes	DSL | Fiber optic | No

Field	Type	Required	Constraints
All other fields	string/int	No	Defaults applied if omitted

POST /predict — Response (PredictionResponse)

Field	Type	Description
churn_prediction	int	0 = stays, 1 = churns
churn_probability	float	Model probability of churn (0.0–1.0)
risk_level	string	Low (<40%), Medium (40–70%), High (>70%)
confidence	string	Confident (prob ≥0.70 or ≤0.30), else Moderate
inference_time_ms	float	End-to-end latency in milliseconds

8. Problems Faced & Solutions

Problem	Root Cause	Solution Applied
Airflow users create command crashed	YAML > block collapsed newlines into single line	Changed to bash -c | literal block; fixed arg parsing
ModuleNotFoundError: sklearn in Airflow	Base airflow:2.8.1 has no ML packages	Created Dockerfile.airflow extending base with all ML packages
sklearn==1.4.2 not installable in Airflow	airflow:2.8.1 uses Python 3.8; sklearn 1.4+ requires Python 3.9+	Pinned all packages to last Python 3.8 compatible versions
ModuleNotFoundError: dags.train	No __init__.py in dags folder; dags.X package lookup fails	Switched to bare imports (import train_v2) after explicit sys.path insertion
Docker MLflow shows no experiments	mlflow.db created separately after filesystem runs; out of sync	Rebuilt SQLite DB from filesystem mlruns/ metadata using Python script
API healthcheck failed	Healthcheck used host port 8005 but runs inside container	Fixed to use internal port 8000 in healthcheck
MLflow artifact paths broke in Docker	Paths hardcoded to /Users/nikesh.../mlruns	sed fix replaced Mac absolute paths with /mlflow/ container paths

9. Advantages & Scope of Improvement

9.1 Advantages

- Full MLOps lifecycle: every stage from raw data to monitored production is automated and reproducible
- Loose coupling: frontend and backend are completely independent, deployable separately
- Training-serving consistency: fitted scaler and encoders are saved and reloaded at inference, preventing skew
- Automated quality gates: CI/CD pipeline enforces F1 ≥ 0.65 before any code merges
- Threshold optimisation: v2 uses the optimal decision threshold per model, not a hardcoded 0.5
- Drift detection: KS-test against training baselines triggers alerts before model degradation becomes visible
- Environment parity: Docker ensures identical behaviour across developer laptops and deployment

9.2 Scope of Improvement

- Replace synthetic dataset with real Telco data; add call records, support history, and payment behaviour features
- Integrate XGBoost / LightGBM — consistently top performers on tabular churn datasets
- Add SHAP per-prediction explanations: 'This customer is at risk because of Month-to-month contract (+28%)'
- Implement POST /feedback endpoint to receive actual churn outcomes and compute real-world model decay
- Add API authentication (HTTPBearer) and rate limiting (slowapi) before any public deployment
- Replace SQLite with PostgreSQL for Airflow and MLflow to support parallel execution and production scale
- Add A/B testing: route 10% of traffic to new model version and compare real-world performance before full cutover

10. Conclusion

ChurnGuard demonstrates a complete, production-ready MLOps application built entirely on local infrastructure without cloud dependencies. All course requirements are implemented: data ingestion and validation pipeline (Airflow), data and model versioning (Git + DVC), experiment tracking with parameters, metrics, and artifacts (MLflow), Prometheus instrumentation with Grafana visualisation, FastAPI model serving with health endpoints, Streamlit frontend with user manual, and Docker Compose multi-container deployment.

The v2 training pipeline improves upon the baseline through GridSearchCV hyperparameter tuning, SMOTE oversampling, and probability threshold optimisation, achieving Test F1=0.525 and ROC-AUC=0.665 — meeting all defined acceptance criteria. Inference latency is consistently under 5 ms, well within the 200 ms SLA.

Every debugging problem encountered during development — from Airflow Python version constraints, to import path resolution, to MLflow database synchronisation — was methodically diagnosed and resolved, building a deeper understanding of MLOps tooling in practice.

— End of Report —